Project Documentation

Credit Card Approval Prediction System — Final Report

1. Abstract

This project delivers a Credit Card Approval Prediction System — a Flask web application backed by a Random Forest classifier trained on 438,557 applicant records merged with 1,048,575 credit history records. Users enter 17 applicant features and receive an instant Approved or Rejected decision with visual feedback via a dedicated result page.

2. Introduction

Manual credit card application review is slow, inconsistent, and prone to bias. This project automates the approval decision using machine learning by merging two datasets — application demographics and credit payment history — and training a classifier that distinguishes low-risk (Approved) from high-risk (Rejected) applicants.

3. Datasets

- **application_record.csv** — 438,557 rows, 18 columns. Applicant demographics, income, employment, housing, contact flags. 134,203 missing OCCUPATION_TYPE values filled with 'Unknown'.
- **credit_record.csv** — 1,048,575 rows, 3 columns (ID, MONTHS_BALANCE, STATUS). STATUS codes 1–5 indicate late/default payments and are used to derive TARGET=1.

4. Target Variable

Applicants with any STATUS code in [1, 2, 3, 4, 5] across their credit history are labelled TARGET=1 (Rejected / High Risk). All others are TARGET=0 (Approved / Low Risk). The two datasets are joined on the ID column.

5. Model Training

Four models were trained and compared using a full scikit-learn Pipeline (ColumnTransformer + classifier): Logistic Regression, Decision Tree, Random Forest, and XGBoost. Random Forest (n_estimators=100) achieved the highest Accuracy (~89%), F1-Score (~87%), and ROC-AUC (~95%) and was selected as the production model.

6. System Architecture

Offline: train_model.py merges CSVs, derives TARGET, trains models, saves the full Pipeline to best_credit_card_model.pkl and column order to feature_columns.pkl. Online: Flask app.py loads the Pipeline once at startup, accepts 17 form fields via POST /predict, builds a DataFrame in the correct column order, and passes it directly to pipeline.predict(). Results are rendered on result.html with a green-tick (Approved) or red-cross (Rejected) card.

7. Key Implementation Details

- Full Pipeline pkl — preprocessing (StandardScaler + OneHotEncoder) is bundled inside the pkl, so app.py feeds raw string form values without any manual encoding.
- Friendly input mode — JavaScript converts age (years) to DAYS_BIRTH (negative days) and experience (years) to DAYS_EMPLOYED before form submission.
- Raw mode toggle — technical users can switch to direct negative-day input.
- handle_unknown='ignore' in OneHotEncoder safely handles new occupation values.
- Error handling — all exceptions caught; result.html renders an error card (■).
- Deployment — render.yaml + Procfile configure Render free-tier deployment (gunicorn app:app, Python 3.12.3).

8. Testing Summary

10 test cases covering both prediction outcomes, friendly/raw input conversion, toggle mode, HTML5 validation, server-side error handling, OHE unknown handling, and navigation. All 10 passed.

9. Deployment

Application deployed on Render (free tier). Build: pip install -r requirements.txt. Start: gunicorn app:app. Runtime: Python 3.12.3 (runtime.txt). Configuration: render.yaml (service name: credit-card-approval-prediction).

10. Conclusion & Future Scope

The system successfully automates credit card approval decisions using ML on real-world credit history data. Future enhancements: probability score display, SHAP explainability for feature importance, API endpoint for bank system integration, and retraining pipeline triggered by new data uploads.

11. References

- Dataset: Dataset/application_record.csv and credit_record.csv
- scikit-learn 1.8.0 documentation — scikit-learn.org
- Flask 3.1.3 documentation — flask.palletsprojects.com
- Render deployment docs — render.com/docs
- joblib documentation — joblib.readthedocs.io